

Develop code that makes use of assertions, and distinguish appropriate from inappropriate uses of assertions.

Two forms of the assert statement

1. Usual form

An assert statement has two parts separated by a colon. The boolean condition must be true for execution to continue. If it is false, an `AssertionError` is thrown, which terminates execution and display the message string. Some examples:

```
assert jobQueue.size() == 0 : "process: queue should have been empty.";
```

```
assert connector != null : "merge: Connector null for " + rel;
```

When asserts are enabled, the assert statement checks the condition (queue empty, connector is not null, etc) which must be true for the program to function correctly. If it's true, execution continues. If connector is null (expression is false), an exception containing the message is thrown. This message is for the programmer, so it doesn't have to be user friendly.

2. Abbreviated form

The simplest form the assert statement specifies only a boolean expression that must be true. This is ok when there's not much to say, or the likelihood of failing seems so remote it isn't worth the extra typing:

```
assert n > 0;
```

Enabling assertions at runtime

Assertion checking defaults to off at runtime. You should always turn them on.

assert statements are removed by class loader:

```
java MyProg
```

assert statements are executed:

```
java -enableassertions MyProg
```

Short form for allowing assertions:

```
java -ea MyProg
```

To disable assertions at various granularities, use:

```
java -disableassertions MyProg
```

or

```
java -da MyProg
```

You specify the granularity with the arguments that you provide to the switch:

- no arguments

Enables or disables assertions in all classes except system classes.

- `packageName...`

Enables or disables assertions in the named package and any subpackages.

- ...

Enables or disables assertions in the unnamed package in the current working directory.

- `className`

Enables or disables assertions in the named class

For example, the following command runs a program, `BatTutor`, with assertions enabled in only package `com.wombat.fruitbat` and its subpackages:

```
java -ea:com.wombat.fruitbat... BatTutor
```

If a single command line contains multiple instances of these switches, they are processed in order before loading any classes. For example, the following command runs the `BatTutor` program with assertions enabled in package `com.wombat.fruitbat` but disabled in class `com.wombat.fruitbat.Brickbat`:

```
java -ea:com.wombat.fruitbat... -da:com.wombat.fruitbat.Brickbat BatTutor
```

The above switches apply to all class loaders. With one exception, they also apply to system classes (which do not have an explicit class loader). The exception concerns the switches with no arguments, which (as indicated above) do not apply to system classes. This behavior makes it easy to enable asserts in all classes except for system classes, which is commonly desirable.

To enable assertions in all system classes, use a different switch: `-enablesystemassertions`, or `-esa`. Similarly, to disable assertions in system classes, use `-disablesystemassertions`, or `-dsa`.

For example, the following command runs the `BatTutor` program with assertions enabled in system classes, as well as in the `com.wombat.fruitbat` package and its subpackages:

```
java -esa -ea:com.wombat.fruitbat... BatTutor
```

The assertion status of a class (enabled or disabled) is set at the time it is initialized, and does not change.

To compile using J2SE 1.4:

```
javac -source 1.4 AssertTest.java
```

To compile using J2SE 5.0 (assertions are recognized by default):

```
javac AssertTest.java
```

or (explicitly define source code version)

```
javac -source 1.5 AssertTest.java
```

To test assertions during runtime (both J2SE 1.4 and J2SE 5.0):

```
java -ea AssertTest
```

```
class AssertTest {
    public static void main(String[] args) {
        // The following assert statement will stop execution
        // with a message if assertions are turned on.
        assert false : "Assertions are turned on.";

        // The following statement will only be printed if
        // assertions are turned off because assertions
        // were not allowed at run time by the -ea parameter.
        System.out.println("Assertions are not active.");
    }
}
```

There are a few situations where you should not use assertions:

1. Do not use assertions for argument checking in public methods.

Argument checking is typically part of the published specifications (or contract) of a method, and these specifications must be obeyed whether assertions are enabled or disabled. Another problem with using assertions for argument checking is that erroneous arguments should result in an appropriate runtime exception (such as `IllegalArgumentException`, `IndexOutOfBoundsException`, or `NullPointerException`). An assertion failure will not throw an appropriate exception.

2. Do not use assertions to do any work that your application requires for correct operation.

Because assertions may be disabled, programs MUST NOT assume that the boolean expression contained in an assertion will be evaluated. Violating this rule has dire consequences. For example, suppose you wanted to remove all of the null elements from a list `names`, and knew that the list contained one or more nulls. It would be wrong to do this:

```
// Broken! - action is contained in assertion
assert names.remove(null);
```

The program would work fine when asserts were enabled, but would fail when they were disabled, as it would no longer remove the null elements from the list. The correct idiom is to perform the action before the assertion and

then assert that the action succeeded:

```
// Fixed - action precedes assertion
boolean nullsRemoved = names.remove(null);
assert nullsRemoved; // Runs whether or not asserts are enabled
```

As a rule, the expressions contained in assertions should be free of side effects: evaluating the expression should not affect any state that is visible after the evaluation is complete. One exception to this rule is that assertions can modify state that is used only from within other assertions.

There are many situations where it is good to use assertions:

1. Internal Invariants

Before assertions were available, many programmers used comments to indicate their assumptions concerning a program's behavior. You should now use an assertion whenever you would have written a comment that asserts an invariant:

```
if (i % 3 == 0) {
    ...
} else {
    if (i % 3 == 1) {
        ...
    } else {
        assert i % 3 == 2 : i;
        ...
    }
}
```

Another good candidate for an assertion is a switch statement with no default case. The absence of a default case typically indicates that a programmer believes that one of the cases will always be executed. The assumption that a particular variable will have one of a small number of values is an invariant that should be checked with an assertion:

```
default:
    assert false : suit;
```

If the `suit` variable takes on another value and assertions are enabled, the assert will fail and an `AssertionError` will be thrown.

An acceptable alternative is:

```
default:
    throw new AssertionError(suit);
```

This alternative offers protection even if assertions are disabled, but the extra protection adds no cost: the throw statement won't execute unless the program has failed. Moreover, the alternative is legal under some circumstances where the `assert` statement is not. If the enclosing method returns a value, each case in the switch statement contains a return statement, and no return statement follows the switch statement, then it would cause a syntax error to add a default case with an assertion. (The method would return without a value if no case matched and assertions were disabled.)

2. Control-Flow Invariants

Place an assertion at any location you assume will not be reached. The assertions statement to use is:

```
assert false;
```

Code now reads:

```
void foo() {  
    for (...) {  
        if (...) return;  
    }  
    assert false; // Execution should never reach this point!  
}
```

3. Preconditions, Postconditions, and Class Invariants

While the `assert` construct is not a full-blown design-by-contract facility, it can help support an informal design-by-contract style of programming.

Do not use assertions to check the parameters of a public method. An `assert` is inappropriate because the method guarantees that it will always enforce the argument checks. It must check its arguments whether or not assertions are enabled. Further, the `assert` construct does not throw an exception of the specified type. It can throw only an `AssertionError`.

You can, however, use an assertion to test a nonpublic method's precondition that you believe will be true no matter what a client does with the class.

You can test postcondition with assertions in both public and nonpublic methods.

A class invariant is a type of internal invariant that applies to every instance of a class at all times, except when an instance is in transition from one consistent state to another. A class invariant can specify the relationships among multiple attributes, and should be true before and after any method completes. For example, suppose you implement a balanced tree data structure of some sort. A class invariant might be that the tree is balanced and properly ordered.

The assertion mechanism does not enforce any particular style for checking invariants. It is sometimes convenient, though, to combine the expressions that check required constraints into a single internal method that can be called by assertions. Continuing the balanced tree example, it might be appropriate to implement a private method that checked that the tree was indeed balanced as per the dictates of the data structure:

```
// Returns true if this tree is properly balanced  
private boolean balanced() {  
    ...  
}
```

Because this method checks a constraint that should be true before and after any method completes, each public method and constructor should contain the following line immediately prior to its return:

```
assert balanced();
```

It is generally unnecessary to place similar checks at the head of each public method unless the data structure is implemented by native methods. In this case, it is possible that a memory corruption bug could corrupt a "native peer" data structure in between method invocations. A failure of the assertion at the head of such a method would indicate that such memory corruption had occurred. Similarly, it may be advisable to include class invariant checks at the heads of methods in classes whose state is modifiable by other classes.

In order for the `javac` compiler to accept code containing assertions, you must use the `-source 1.4` command-line option as in this example:

```
javac -source 1.4 MyClass.java
```

This flag is necessary so as not to cause source compatibility problems.

NOTE, in Java 5.0 the compiler accepts assertions BY DEFAULT:

```
javac MyClass.java
```

In both Java 5.0, as in Java 1.4 assertions are disabled by default at runtime. You need explicitly to turn them on.

At a runtime you can check if the program is running with enabled assertions using the following code:

```
boolean assertsEnabled = false;  
assert assertsEnabled = true; // Intentional side-effect !!!  
// Now 'assertsEnabled' is set to the correct value
```

[Prev](#)

Develop code that implements all forms of loops and iterators, including the use of for, the enhanced for loop (for-each), do, while, labels, break, and continue; and explain the values taken by loop counter variables during and after loop execution.



SCDJWS 1.4
Quiz



ICAD
Study Guide



Magnet
Mocker

SCWCD 1.4
Study Guide

[Up](#)

[Home](#)

SCBCD 1.3
Study Guide

SCDJWS 1.4
Study Guide

[Next](#)

Develop code that makes use of exceptions and exception handling clauses (try, catch, finally), and declares methods and overriding methods that throw exceptions.

IBM Test 287
Study Guide